Đây là notebook đồng hành cho cuốn sách Deep Learning with Python, Third Edition. Để dễ đọc, notebook này chỉ chứa các khối mã có thể chạy được và các tiêu đề phần, lược bỏ các đoạn văn bản, hình ảnh và mã giả trong sách.

Nếu bạn muốn theo dõi những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản in hoặc bản điện tử của cuốn sách.

Nội dung sách có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

Hiện mã

Sinh văn bản (Text generation)

Sơ lược lịch sử về tạo chuỗi (sequence generation)

Huấn luyện một mô hình mini-GPT

```
import os

# Free up more GPU memory on the Jax and TensorFlow backends.
os.environ["XLA_PYTHON_CLIENT_MEM_FRACTION"] = "1.00"
```

```
import keras
import pathlib

extract_dir = keras.utils.get_file(
    fname="mini-c4",
    origin=(
        "https://hf.co/datasets/mattdangerw/mini-c4/resolve/main/mini-c4.zip"
    ),
    extract=True,
)
extract_dir = pathlib.Path(extract_dir) / "mini-c4"
```

```
with open(extract_dir / "shard0.txt", "r") as f:
    print(f.readline().replace("\\n", "\n")[:100])
```

```
import keras_hub
import numpy as np

vocabulary_file = keras.utils.get_file(
    origin="https://hf.co/mattdangerw/spiece/resolve/main/vocabulary.proto",
)
tokenizer = keras_hub.tokenizers.SentencePieceTokenizer(vocabulary_file)
```

```
tokenizer.tokenize("The quick brown fox.")
```

```
tokenizer.detokenize([450, 4996, 17354, 1701, 29916, 29889])
```

```
import tensorflow as tf

batch_size = 64
sequence_length = 256
suffix = np.array([tokenizer.token_to_id("<|endoftext|>")])

def read_file(filename):
    ds = tf.data.TextLineDataset(filename)
    ds = ds.map(lambda x: tf.strings.regex_replace(x, r"\\n", "\n"))
    ds = ds.map(tokenizer, num_parallel_calls=8)
```

```
    return ds.map(lambda x: tf.concat([x, suffix], -1))

files = [str(file) for file in extract_dir.glob("*.txt")]
ds = tf.data.Dataset.from_tensor_slices(files)
ds = ds.interleave(read_file, cycle_length=32, num_parallel_calls=32)
ds = ds.rebatch(sequence_length + 1, drop_remainder=True)
ds = ds.map(lambda x: (x[:-1], x[1:]))
ds = ds.batch(batch_size).prefetch(8)
```

```
num_batches = 58746
num_val_batches = 500
num_train_batches = num_batches - num_val_batches
val_ds = ds.take(num_val_batches).repeat()
train_ds = ds.skip(num_val_batches).repeat()
```

Xây dựng mô hình

```
from keras import layers

class TransformerDecoder(keras.Layer):
    def __init__(self, hidden_dim, intermediate_dim, num_heads):
        super().__init__()
        key_dim = hidden_dim // num_heads
        self.self_attention = layers.MultiHeadAttention(
            num_heads, key_dim, dropout=0.1
        )
        self.self_attention_layernorm = layers.LayerNormalization()
        self.feed_forward_1 = layers.Dense(intermediate_dim, activation="relu")
        self.feed_forward_2 = layers.Dense(hidden_dim)
        self.feed_forward_layernorm = layers.LayerNormalization()
        self.dropout = layers.Dropout(0.1)

    def call(self, inputs):
        residual = x = inputs
        x = self.self_attention(query=x, key=x, value=x, use_causal_mask=True)
        x = self.dropout(x)
        x = x + residual
        x = self.self_attention_layernorm(x)
        residual = x
        x = self.feed_forward_1(x)
        x = self.feed_forward_2(x)
        x = self.dropout(x)
        x = x + residual
        x = self.feed_forward_layernorm(x)
        return x
```

```
from keras import ops

class PositionalEmbedding(keras.Layer):
    def __init__(self, sequence_length, input_dim, output_dim):
        super().__init__()
        self.token_embeddings = layers.Embedding(input_dim, output_dim)
        self.position_embeddings = layers.Embedding(sequence_length, output_dim)

    def call(self, inputs, reverse=False):
        if reverse:
            token_embeddings = self.token_embeddings.embeddings
            return ops.matmul(inputs, ops.transpose(token_embeddings))
        positions = ops.cumsum(ops.ones_like(inputs), axis=-1) - 1
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions
```

```
keras.config.set_dtype_policy("mixed_float16")

vocab_size = tokenizer.vocabulary_size()
hidden_dim = 512
intermediate_dim = 2056
num_heads = 8
num_layers = 8

inputs = keras.Input(shape=(None,), dtype="int32", name="inputs")
embedding = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)
```

```
x = embedding(inputs)
x = layers.LayerNormalization()(x)
for i in range(num_layers):
    x = TransformerDecoder(hidden_dim, intermediate_dim, num_heads)(x)
outputs = embedding(x, reverse=True)
mini_gpt = keras.Model(inputs, outputs)
```

Tiền huấn luyện mô hình (Pretraining)

```
class WarmupSchedule(keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self):
        self.rate = 2e-4
        self.warmup_steps = 1_000.0

    def __call__(self, step):
        step = ops.cast(step, dtype="float32")
        scale = ops.minimum(step / self.warmup_steps, 1.0)
        return self.rate * scale
```

```
import matplotlib.pyplot as plt

schedule = WarmupSchedule()
x = range(0, 5_000, 100)
y = [ops.convert_to_numpy(schedule(step)) for step in x]
plt.plot(x, y)
plt.xlabel("Train Step")
plt.ylabel("Learning Rate")
plt.show()
```

```
# ⚠NOTE⚠: If you can run the following with a Colab Pro GPU, we suggest you
# do so. This fit() call will take many hours on free tier GPUs. You can also
# reduce steps_per_epoch to try the code with a less trained model.
```

```
num_epochs = 8
steps_per_epoch = num_train_batches // num_epochs
validation_steps = num_val_batches

mini_gpt.compile(
    optimizer=keras.optimizers.Adam(schedule),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
mini_gpt.fit(
    train_ds,
    validation_data=val_ds,
    epochs=num_epochs,
    steps_per_epoch=steps_per_epoch,
    validation_steps=validation_steps,
)
```

Giải mã sinh văn bản (Generative decoding)

```
def generate(prompt, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    for _ in range(max_length - prompt_length):
        prediction = mini_gpt(ops.convert_to_numpy([tokens]))
        prediction = ops.convert_to_numpy(prediction[0, -1])
        tokens.append(np.argmax(prediction).item())
    return tokenizer.detokenize(tokens)
```

```
prompt = "A piece of advice"
generate(prompt)
```

```
def compiled_generate(prompt, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    tokens = tokens + [0] * (max_length - prompt_length)
    for i in range(prompt_length, max_length):
        prediction = mini_gpt.predict(np.array([tokens]), verbose=0)
```

```
        prediction = prediction[0, i - 1]
        tokens[i] = np.argmax(prediction).item()
    return tokenizer.detokenize(tokens)
```

```
import timeit
tries = 10
timeit.timeit(lambda: compiled_generate(prompt), number=tries) / tries
```

Các chiến lược lấy mẫu (Sampling strategies)

```
def compiled_generate(prompt, sample_fn, max_length=64):
    tokens = list(ops.convert_to_numpy(tokenizer(prompt)))
    prompt_length = len(tokens)
    tokens = tokens + [0] * (max_length - prompt_length)
    for i in range(prompt_length, max_length):
        prediction = mini_gpt.predict(np.array([tokens]), verbose=0)
        prediction = prediction[0, i - 1]
        next_token = ops.convert_to_numpy(sample_fn(prediction))
        tokens[i] = np.array(next_token).item()
    return tokenizer.detokenize(tokens)
```

```
def greedy_search(preds):
    return ops.argmax(preds)

compiled_generate(prompt, greedy_search)
```

```
def random_sample(preds, temperature=1.0):
    preds = preds / temperature
    return keras.random.categorical(preds[None, :], num_samples=1)[0]
```

```
compiled_generate(prompt, random_sample)
```

```
from functools import partial
compiled_generate(prompt, partial(random_sample, temperature=2.0))
```

```
compiled_generate(prompt, partial(random_sample, temperature=0.8))
```

```
compiled_generate(prompt, partial(random_sample, temperature=0.2))
```

```
def top_k(preds, k=5, temperature=1.0):
    preds = preds / temperature
    top_preds, top_indices = ops.top_k(preds, k=k, sorted=False)
    choice = keras.random.categorical(top_preds[None, :], num_samples=1)[0]
    return ops.take_along_axis(top_indices, choice, axis=-1)
```

```
compiled_generate(prompt, partial(top_k, k=5))
```

```
compiled_generate(prompt, partial(top_k, k=20))
```

```
compiled_generate(prompt, partial(top_k, k=5, temperature=0.5))
```

Sử dụng một mô hình LLM đã được huấn luyện sẵn

Sinh văn bản với mô hình Gemma

```
import kagglehub

kagglehub.login()
```

```
gemma_lm = keras_hub.models.CausalLM.from_preset(
    "gemma3_1b",
    dtype="float32",
)
```

```
gemma_lm.summary(line_length=80)
```

```
gemma_lm.compile(sampler="greedy")
gemma_lm.generate("A piece of advice", max_length=40)
```

```
gemma_lm.generate("How can I make brownies?", max_length=40)
```

```
gemma_lm.generate(
    "The following brownie recipe is easy to make in just a few "
    "steps.\n\nYou can start by",
    max_length=40,
)
```

```
gemma_lm.generate(
    "Tell me about the 542nd president of the United States.",
    max_length=40,
)
```

Tinh chỉnh theo chỉ dẫn (Instruction fine-tuning)

```
import json

PROMPT_TEMPLATE = """[instruction]\n{}[end]\n[response]\n"""
RESPONSE_TEMPLATE = """{}[end]"""

dataset_path = keras.utils.get_file(
    origin=(
        "https://hf.co/datasets/databricks/databricks-dolly-15k/"
        "resolve/main/databricks-dolly-15k.jsonl"
    ),
)
data = {"prompts": [], "responses": []}
with open(dataset_path) as file:
    for line in file:
        features = json.loads(line)
        if features["context"]:
            continue
        data["prompts"].append(PROMPT_TEMPLATE.format(features["instruction"]))
        data["responses"].append(RESPONSE_TEMPLATE.format(features["response"]))
```

```
data["prompts"][0]
```

```
data["responses"][0]
```

```
ds = tf.data.Dataset.from_tensor_slices(data).shuffle(2000).batch(2)
val_ds = ds.take(100)
train_ds = ds.skip(100)
```

```
preprocessor = gemma_lm.preprocessor
preprocessor.sequence_length = 512
batch = next(iter(train_ds))
x, y, sample_weight = preprocessor(batch)
x["token_ids"].shape
```

```
x["padding_mask"].shape
```

```
y.shape
```

```
sample_weight.shape
```

```
x["token_ids"][0, :5], y[0, :5]
```

Thích nghi thứ hạng thấp (Low-Rank Adaptation - LoRA)

```
```

```
gemma_lm.backbone.enable_lora(rank=8)
```

```
gemma_lm.summary(line_length=80)
```

```
gemma_lm.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(5e-5),
    weighted_metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
gemma_lm.fit(train_ds, validation_data=val_ds, epochs=1)
```

```
gemma_lm.generate(
    "[instruction]\nHow can I make brownies?[end]\n"
    "[response]\n",
    max_length=512,
)
```

```
gemma_lm.generate(
    "[instruction]\nWhat is a proper noun?[end]\n"
    "[response]\n",
    max_length=512,
)
```

```
gemma_lm.generate(
    "[instruction]\nWho is the 542nd president of the United States?[end]\n"
    "[response]\n",
    max_length=512,
)
```

Tiến xa hơn với các mô hình ngôn ngữ lớn (LLMs)

Học tăng cường từ phản hồi của con người (RLHF)

Sử dụng một chatbot được huấn luyện với RLHF

```
# ⚠NOTE⚠: If you are running on the free tier Colab GPUs, you will need to
# restart your runtime and run the notebook from here to free up memory for
# this 4 billion parameter model.
import os

os.environ["KERAS_BACKEND"] = "jax"
# Free up more GPU memory on the Jax and TensorFlow backends.
os.environ["XLA_PYTHON_CLIENT_MEM_FRACTION"] = "1.00"

import keras
import keras_hub
import kagglehub
import numpy as np

kagglehub.login()
```

```
gemma_lm = keras_hub.models.CausalLM.from_preset(
    "gemma3_instruct_4b",
    dtype="bfloat16",
)
```

```
PROMPT_TEMPLATE = """<start_of_turn>user
{}<end_of_turn>
<start_of_turn>model
"""
```

```
prompt = "Why can't you assign values in Jax tensors? Be brief!"
gemma_lm.generate(PROMPT_TEMPLATE.format(prompt), max_length=512)
```

```
prompt = "Who is the 542nd president of the United States?"
gemma_lm.generate(PROMPT_TEMPLATE.format(prompt), max_length=512)
```

Các mô hình đa phương thức (Multimodal LLMs)

```
import matplotlib.pyplot as plt

image_url = (
    "https://github.com/mattdangerw/keras-nlp-scripts/"
    "blob/main/learned-python.png?raw=true"
)
image_path = keras.utils.get_file(origin=image_url)

image = np.array(keras.utils.load_img(image_path))
plt.axis("off")
plt.imshow(image)
plt.show()
```

```
gemma_lm.preprocessor.max_images_per_prompt = 1
gemma_lm.preprocessor.sequence_length = 512
prompt = "What is going on in this image? Be concise!<start_of_image>"
gemma_lm.generate({
    "prompts": PROMPT_TEMPLATE.format(prompt),
    "images": [image],
})
```

```
prompt = "What is the snake wearing?<start_of_image>"
gemma_lm.generate({
    "prompts": PROMPT_TEMPLATE.format(prompt),
    "images": [image],
})
```

Các mô hình nền tảng (Foundation models)

Tạo lập tăng cường tra cứu (Retrieval Augmented Generation - RAG)

Các mô hình "Suy luận" (Reasoning models)

```
prompt = """Judy wrote a 2-page letter to 3 friends twice a week for 3 months.
How many letters did she write?
Be brief, and add "ANSWER:" before your final answer."""

gemma_lm.compile(sampler="random")
```

```
gemma_lm.generate(PROMPT_TEMPLATE.format(prompt))
```

```
gemma_lm.generate(PROMPT_TEMPLATE.format(prompt))
```

Where are LLMs heading next?